

Subagent 架构对比报告

ByteDance DeerFlow 2.0 `task tool` vs EphemeralOS `run_subagent` + background subsystem
日期: 2026-04-08 · 作者: Yifan Xu · 基于源码一手验证 (file:line 可追溯)

执行摘要

两个系统在同一个问题——"LLM 不应浪费 token 主动轮询 subagent"——上做了相反的架构抽象：

- DeerFlow 2.0: 把 subagent 包装成同步函数调用，所有异步性藏在 host 的 `asyncio.sleep(5)` 循环后。LLM 认知负担最小，但失去调度能力。
- EphemeralOS: 把 subagent 当成 OS 级 process，给 LLM 一整套生命周期调度原语 (`run / check / wait / cancel` + `bg_N` 别名)。工程复杂度高，但换来真正的并发、可观测、可恢复。

对于 subagent swarm 场景，EphemeralOS 在 6/6 核心维度上领先；DeerFlow 2.0 从架构层面就不是 swarm 平台。

1. 核心设计哲学

维度	DeerFlow 2.0	EphemeralOS
核心隐喻	Subagent = 同步函数调用	Subagent = OS 级 process
LLM 认知负担	最小 (1 个工具返回字符串)	较高 (4 个工具 + 生命周期管理)
调度者	Host (隐藏轮询)	LLM (显式调度原语)
失败域	进程内，重启即丢	DB-backed，可审计可恢复

2. 阻塞语义对比

维度	DeerFlow 2.0	EphemeralOS
LLM 视角	强制阻塞：一次 tool call → 一个最终 string	强制非阻塞： <code>background="always"</code> 立即返回 <code>bg_N</code> 别名
Host 视角	非阻塞 (线程池 + 隐藏 sleep-poll)	非阻塞 (单事件循环 + <code>asyncio.create_task</code>)
等待原语	<code>asyncio.sleep(5)</code> 轮询 (<code>task_tool.py:197</code>)	真正的 <code>asyncio.wait</code> 事件唤醒，零 token 零 CPU (<code>background_tasks.py:181-210</code>)
理论基础	<code>task_tool_improvements.md</code> : "LLM makes only ONE tool call / No wasteful polling"	同一问题的更优解：把 wait 变成真正的事件等待而不是藏起来的 sleep-loop

关键洞察: DeerFlow 对问题的诊断是对的 (LLM 轮询浪费 token)，但它的解法 (host 端 sleep-loop) 只是把 CPU 醒来从 LLM 转移到 host。EphemeralOS 用 asyncio 事件唤醒彻底消除了轮询——这是同一哲学的更彻底实现。

3. 持久化与可恢复性

维度	DeerFlow 2.0	EphemeralOS
存储	<code>_background_tasks: dict + threading.Lock</code> (executor.py:68-70)	DB agent_run 表, 先于 <code>spawn_agent</code> 写入 (run_subagent_tool.py:166-177)
生命周期	父 tool 返回后立即 <code>del</code> (executor.py:528)	<code>finish_run</code> 写完整 display + 压缩快照 + <code>cancellation_reason</code>
重启恢复	❌ 无	✅ 可 eval / replay / retry
父子链接	无	<code>parent_run_id + parent_task_id</code> 成树
Spawn 失败审计	抛异常	仍写 failed 行 (run_subagent_tool.py:187-199)

4. 取消机制

维度	DeerFlow 2.0	EphemeralOS
触发源	仅父 <code>asyncio.Task cancel</code> (用户 abort)	LLM 可显式调用 <code>cancel_background_task</code>
机制	纯协作: <code>cancel_event</code> 在 <code>astream chunk</code> 边界检查	双路径: ① <code>kill_callback</code> 物理 kill 沙箱进程 (绕开 Daytona SDK 防污染) ② pure-Python fallback 用 <code>asyncio.Task.cancel</code>
长 tool 中途打断	❌ 跑完才停	✅ 物理 kill 真正打断
竞态保护	无	<code>_done_callback</code> 显式跳过已 cancelled 的 task, 防止 SDK 物理 kill 后的 <code>exit_code -1</code> "正常"返回覆盖 (background_tasks.py:102-128)
Timeout 默认	900s per-task (config.py:28)	无隐式 per-task timeout (短板)

5. 父子信息带宽

维度	DeerFlow 2.0	EphemeralOS
最终字符串	✅ 最后一条 AIMessage	✅ 最后一条 assistant message
中间进度给 UI	✅ <code>get_stream_writer</code> 流式 <code>task_running</code>	✅ streaming executor + 事件
中间进度给父 LLM	❌ 完全盲到最终返回	✅ LLM 主动 <code>check_background_progress</code> → <code>progress_provider</code> 闭包到 <code>live agent.display_messages</code> , clamp 到 10 条消息 / 2048 字符
完成事件注入父对话	❌ 无	✅ <code>query.py:240-257</code> 每轮 <code>collect_completed</code> → <code>[BACKGROUND task_id COMPLETED]</code> 作为 user 消息追加到 <code>display_messages</code>
增量 reminder	❌ 无	✅ <code>get_reminder_diff</code> 维护 per-task 行索引+时间戳, 只发增量 (<code>background_tasks.py:356-376</code>)

6. 并发与编排

维度	DeerFlow 2.0	EphemeralOS
执行底物	双 <code>ThreadPoolExecutor</code> (scheduler 3 + exec 3)	单事件循环 + <code>asyncio.create_task</code>
并发硬上限	进程级 3, 跨 session 共享	无上限 (14+ DAG 实测通过)
Per-session quota	无	无 (建议补)
多 wave DAG	❌ 严格单层, child 不能再 spawn child	△ 父级多波 fanout 可行, 但 child 仍被硬禁止 spawn
动态 fanout	只能在父的单 turn 内并发	LLM 任意 turn 发起新 worker, 老的继续跑

7. 维度评分表 (1-5 分, 5 最优)

维度	DeerFlow 2.0	EphemeralOS	说明
并发量	2	5	DF 进程级硬顶 3；EOS 无上限
实时性 / 反馈延迟	2	5	DF 父盲；EOS 可实时 peek + 完成事件回注
LLM 认知负担	5	2	DF 1 工具；EOS 需学会调度
持久化 / 可恢复	1	5	DF 内存 dict 重启即丢；EOS DB 行
取消可靠性	2	5	DF 协作；EOS 物理 kill + 防覆盖
可观测性 / 审计	2	5	DF UI 流无持久；EOS DB + reminder diff
父子信息带宽	1	4	DF 单向终值；EOS 双向
上下文洁净度	5	3	DF 只吃最终字符串
可编排性 / DAG	1	4	DF 严格单层
状态共享	4	4	两者都复用父 sandbox；DF 额外拼 skills 到 child prompt
容错 / 重试	2	4	EOS 失败也留审计行
超时治理	3	2	DF 硬 900s；EOS 无 per-task timeout（短板）
工程复杂度（低更好）	5	2	DF 简单直观
Token 效率	4	4	两者 LLM 侧相当，EOS 微优

加权总分（按场景）

场景	DeerFlow 2.0	EphemeralOS
一次性研究任务 / 单 session 小规模	38	34
长期运行 agent 系统 / 生产可运维	28	52
大 fanout DAG / 并行 workers	22	55
Eval-driven 开发（replay 历史 runs）	15	58

8. Swarm 适配性结论

Swarm 意味着：大量、并行、异构、长时间、可观测、可恢复。六个核心维度全部 EphemeralOS 领先：

- 1. 规模上限: EOS 无硬顶 vs DF 进程级 3 worker
- 2. 异构编排: EOS 支持"边跑边开新的"流水线 vs DF 单轮批处理
- 3. 实时观测: EOS 可动态协调（取消 B/C/D 因为 A 已出结果）vs DF 父盲
- 4. 容错恢复: EOS DB 审计可 retry vs DF 内存丢失
- 5. Speculative cancel: EOS 物理 kill vs DF 协作等 chunk 边界

6. LLM 可调度: EOS 4 工具面 vs DF 1 个黑箱

判决: DeerFlow 2.0 从架构层面就不是 swarm 平台。它的每个设计决策——LLM 不可见 task_id、父盲、3 worker 池、内存 dict、协作 cancel、单层禁递归——都在把 swarm 的可能性往死里掐。它是为"LLM 委托一个子任务然后等答案"的同步助手模式设计的。

EphemeralOS 是目前看到的最接近 production-grade subagent swarm 平台的架构，方向正确，剩下是补细节；DeerFlow 则是方向不对，补细节无用。

9. EphemeralOS 待补的 3 块拼图

1. **Per-session 并发 semaphore** (软上限，非硬编码)。目前无上限在小规模没事，swarm 规模上来会击穿下游 API 配额。加在 `BackgroundTaskManager.launch` 入口 `await semaphore.acquire()`。
2. **Per-task 隐式 timeout**。跑飞的 worker 现在只能靠 LLM 主动 cancel 或 session 结束。swarm 里单个 worker 卡住不能拖垮整体——给 `run_subagent` 加可选 `max_seconds` 参数，到点写 `cancellation_reason="timeout"`。
3. **可选多层嵌套递归**。目前硬禁止是为了防爆炸，但真正的 swarm (research → sub-research → reader) 需要这个。改成"可选 + `max_depth` 限制"。

10. 可互相借鉴的点

EphemeralOS → 应该从 DeerFlow 学

- **动态 skills 注入到 subagent system prompt**。`spawn_agent` 目前只吃静态 `agent_def`，父运行时积累的技能/目标 context 漏掉了。建议加 `extra_system_context` 参数。
- **Stale doc 警示**: DeerFlow 的 `task_tool_improvements.md` 说 300s / 2s polling，代码实际是 900s / 5s sleep——保持 docstring 和 config 同步。

DeerFlow → 应该从 EphemeralOS 学

- `asyncio.wait` 代替 `sleep-loop`——同一哲学的更彻底实现
- **DB 持久化 subagent runs**——才能做 eval/replay/RFT
- **物理 kill + done_callback 防覆盖**——纯协作 cancel 在真实生产里是 bug 温床
- **Reminder diff 机制**——如果愿意给父 LLM 开一条带宽的话
- **bg_N LLM 友好别名**——`tool_call_id` 对 LLM 跨 turn 引用不友好

11. 一页版结论

两个系统在同一个问题 ("LLM 不该手动轮询 subagent") 上做了两个相反的抽象决策：

- **DeerFlow 2.0** = "让 subagent 看起来像同步函数，隐藏所有异步性"。简单、小规模、一次性研究任务漂亮；对长期记忆、复杂编排、容错、可 eval 的生产系统不够。

- **EphemeralOS** = "让 subagent 真正变成 OS 级 process，给 LLM 完整的生命周期调度工具"。工程复杂度高，但在 durability、cancellation、可观测性、fanout、父子双向信息流上全面领先。

架构取舍的分水岭是：**你是否相信 LLM 能学会做一个称职的 OS shell**。DeerFlow 不信所以封起来，EphemeralOS 信所以把 primitives 全露出来。两条路都合理，但回答的是不同问题——DeerFlow 优化单次任务的 token 成本和开发简洁性，EphemeralOS 优化长期运行 agent 系统的可运维性和能力上限。

对于 subagent swarm：EphemeralOS 完胜，没有悬念。

报告基于以下源码一手验证：

DeerFlow: `bytedance/deer-flow` ·

`backend/packages/harness/deerflow/tools/builtins/task_tool.py` · `subagents/executor.py`
· `subagents/config.py` · `backend/docs/task_tool_improvements.md`

EphemeralOS: `backend/src/tools/subagent/run_subagent_tool.py` ·

`backend/src/tools/builtins/background/*` ·

`backend/src/engine/runtime/background_tasks.py` · `engine/runtime/agent.py` ·

`engine/core/streaming_executor.py` · `engine/core/query.py` · `agents/registry.py`